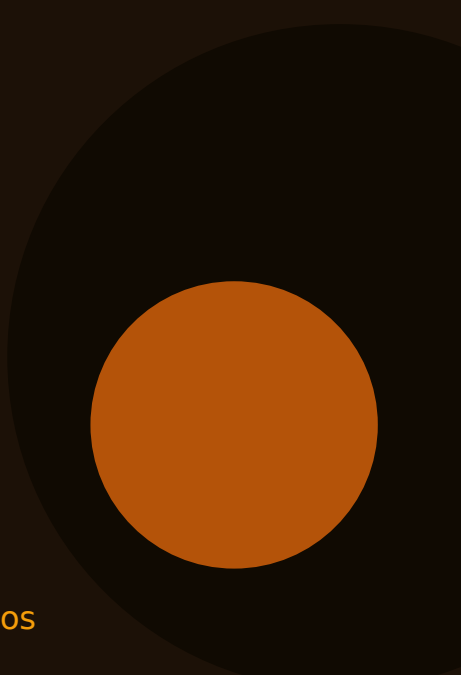




JAVA

IntelliJ & Lógica Condicional

Da IDE ao domínio das condições e dos operadores booleanos

Apostila completa sobre a IDE e a lógica booleana,
com exemplos práticos e explicações passo a passo.

CONTEÚDO DA APOSTILA

- 01 Hello World no IntelliJ
- 02 A Instrução if-then
- 03 = vs == e o Operador NOT !
- 04 AND && e OR ||
- 05 Precedência de Operadores
- 06 Operador Ternário ?:
- 07 Boas Práticas em Lógica Condicional

Sumário

0
1

Hello World no IntelliJ

A IDE, os modificadores de acesso e o método main

0
2

A Instrução if-then

Como o programa passa a tomar decisões

0
3

= vs == e o Operador NOT !

Atribuição, igualdade e negação lógica

0
4

AND && e OR ||

Combinando condições e o short-circuit evaluation

0
5

Precedência de Operadores

A ordem em que o Java avalia uma expressão

0
6

Operador Ternário ?:

O atalho de uma linha para if-else simples

0
7

Boas Práticas em Lógica Condicional

Convenções, atalhos de IDE e erros para evitar

01

Hello World no IntelliJ

Configurando a IDE, entendendo modificadores e o ponto de entrada da aplicação

Por que sair do JShell e usar uma IDE

O IntelliJ IDEA, da JetBrains, é hoje a IDE mais adotada para desenvolvimento em Java, disponível em uma versão Community gratuita e em uma versão Ultimate paga. A diferença em relação ao JShell é estrutural: enquanto o JShell executa uma linha por vez, o IntelliJ organiza o código em projetos com múltiplos arquivos, sinaliza erros de compilação enquanto você digita e oferece recursos como autocomplete inteligente, refatoração automática e um debugger completo.

```
// Arquivo: SistemaAcademia.java
public class SistemaAcademia {

    // Método main: ponto de entrada de todo programa Java
    public static void main(String[] args) {
        System.out.println("Sistema da academia iniciado!");
    }
}
```

Modificadores de acesso

Modificadores de acesso definem quem pode enxergar e usar uma classe, método ou variável. O Java trabalha com quatro níveis: **public** (visível de qualquer lugar), **protected** (visível na própria classe, no pacote e em subclasses), **package-private** (o padrão, quando nenhum modificador é escrito — visível apenas dentro do mesmo pacote) e **private** (restrito à própria classe). Para que a JVM consiga encontrar o ponto de entrada do programa, tanto a classe principal quanto o método main precisam ser public.

DICA — Atalhos que aceleram o dia a dia

psvm seguido de Tab expande automaticamente para `public static void main(String[] args) {}`; sout seguido de Tab expande para `System.out.println()`. Ctrl+Space aciona o autocomplete e Alt+Enter sugere correções automáticas para o código sob o cursor.

02

A Instrução if-then

Ensinando o programa a executar código apenas quando uma condição é satisfeita

Dando ao programa o poder de decidir

Sem estrutura condicional, um programa sempre executaria a mesma sequência de passos, na mesma ordem, todas as vezes. A lógica condicional muda isso: permite que um bloco de código só rode quando uma condição específica for verdadeira. O if é a construção mais elementar de controle de fluxo, presente em praticamente qualquer linguagem de programação moderna.

```
boolean mensalidadeEmDia = false;

// FORMA RECOMENDADA – sempre use chaves {}:
if (mensalidadeEmDia == false) {
    System.out.println("Acesso negado!");
    System.out.println("Regularize sua mensalidade.");
}

// Com else – roda quando a condição é false:
if (mensalidadeEmDia == false) {
    System.out.println("Catraca bloqueada.");
} else {
    System.out.println("Catraca liberada!");
}
```

ATENÇÃO

Use chaves {} no if mesmo quando houver apenas uma instrução dentro dele. Sem elas, apenas a linha imediatamente seguinte pertence ao if — qualquer linha extra adicionada depois passará a executar sempre, de forma incondicional, criando bugs difíceis de perceber.

03

= vs == e o Operador NOT !

A diferença entre definir um valor e comparar dois valores

Atribuição não é igualdade

Confundir `=` com `==` está entre os erros mais frequentes de quem começa a programar. Um único sinal de igual (`=`) é o operador de atribuição: ele define o valor de uma variável. Dois sinais (`==`) formam o operador de igualdade: ele compara dois valores e devolve um boolean. O detalhe perigoso é que o IntelliJ não acusa erro ao usar `=` dentro de um `if` envolvendo boolean, o que torna essa confusão especialmente difícil de detectar.

```
boolean portaTrancada = true; // = ATRIBUI true a portaTrancada

// == COMPARA portaTrancada com true:
if (portaTrancada == true) { System.out.println("Aguarde a liberação"); }

// ERRADO - atribui false e testa o resultado da atribuição:
// if (portaTrancada = false) { } // compila, mas o comportamento é outro!

// CORRETO e idiomático em Java:
if (!portaTrancada) { System.out.println("Pode entrar"); }
if (portaTrancada) { System.out.println("Aguarde a liberação"); }
```

REGRA

Prefira `if (!variavel)` e `if (variavel)` em vez de `if (variavel == false)` e `if (variavel == true)`. É mais curto, elimina o risco de trocar `=` por engano e é a forma que a comunidade Java considera idiomática.

04

AND && e OR ||

Combinando duas ou mais condições em uma única expressão booleana

&& exige que tudo seja verdadeiro

O operador **&&** (AND lógico) só devolve **true** quando as duas condições envolvidas também forem verdadeiras; basta uma delas ser **false** para o resultado inteiro virar **false**. O Java aplica aqui o chamado short-circuit evaluation: se a primeira condição já for **false**, a segunda nem chega a ser avaliada, porque o resultado final já está decidido — um ganho de eficiência automático.

|| basta uma ser verdadeira

Já o operador **||** (OR lógico) devolve **true** assim que pelo menos uma das condições for verdadeira, só chegando a **false** quando ambas forem **false**. O short-circuit também se aplica aqui, na direção oposta: se a primeira condição já for **true**, a segunda não precisa ser avaliada.

```
// AND (&&) – as duas precisam ser true:
boolean temCadastroAtivo = true, pagamentoEmDia = true, temAtestadoValido = false;

if (temCadastroAtivo && pagamentoEmDia) {
    System.out.println("Pode entrar na academia!"); // imprime
}
if (pagamentoEmDia && temAtestadoValido) {
    System.out.println("Pode fazer musculação!"); // NÃO imprime
}

// OR (||) – basta uma ser true:
boolean temPlanoMensal = false, temPlanoAnual = true;
if (temPlanoMensal || temPlanoAnual) {
    System.out.println("Pode acessar a academia!"); // imprime
}
```

05

Precedência de Operadores

Entendendo em que ordem o Java avalia uma expressão complexa

Por que a ordem de avaliação importa

Quando uma expressão reúne vários operadores, o Java precisa decidir qual deles é resolvido primeiro — essa ordem é a precedência de operadores. De forma simplificada, a sequência vai de parênteses, passando por operadores unários (!, ++, --), multiplicativos (*, /, %), aditivos (+, -), relacionais (<, >, <=, >=), igualdade (==, !=), AND lógico (&&), OR lógico (||), ternário (?:) e, por último, atribuição (=).

```
// Sem parênteses – o resultado pode surpreender:
int totalAulas = 2 + 3 * 4; // 14, não 20 – * é resolvido antes de +

// Com parênteses – a ordem desejada fica garantida:
int totalAulas2 = (2 + 3) * 4; // 20

// && tem precedência maior que ||:
boolean x = true || false && false; // true (&& antes de ||)
boolean y = (true || false) && false; // false – parêntese muda tudo

// Calculando o valor da mensalidade com matrícula:
double valorBase = 120.00, taxaMatricula = 30.00;
double totalPrimeiroMes = (valorBase + taxaMatricula) * 1.00; // 150.0
double resto = totalPrimeiroMes % 50.00; // 0.0
boolean valorRedondo = (resto == 0.0); // true
```

DICA

Sempre que && e || aparecerem juntos na mesma expressão, use parênteses explícitos mesmo quando não forem estritamente necessários. O ganho em clareza para quem lê o código depois vale bem mais do que economizar dois caracteres.

06

Operador Ternário ?:

O único operador de três operandos da linguagem

Um atalho elegante para if-else simples

O operador ternário é o único da linguagem que trabalha com três operandos, e por isso é chamado formalmente de operador condicional. Ele funciona como um atalho para situações de if-else que só precisam escolher entre dois valores, seguindo a sintaxe: `variavel = (condição) ? valorSeVerdadeiro : valorSeFalso`.

```
int idade = 16;

// if-else tradicional:
String categoria;
if (idade >= 18) { categoria = "Adulto"; }
else { categoria = "Menor de idade"; }

// Ternário – mesmo resultado, mais direto:
String categoria2 = (idade >= 18) ? "Adulto" : "Menor de idade";

// Anti-padrão: usar o ternário para produzir um boolean:
boolean isAdulto = (idade >= 18) ? true : false; // redundante!

// Forma correta – a condição já É o boolean:
boolean isAdulto2 = (idade >= 18);
```

DICA

O ternário funciona muito bem para expressões curtas de uma linha. Quando a lógica envolve várias ações, prefira o if-else tradicional — e evite ternários aninhados, que prejudicam bastante a legibilidade.

07

Boas Práticas em Lógica Condicional

Convenções da linguagem e os erros mais comuns ao trabalhar com condições

Erros que vale a pena internalizar para nunca cometer

Alguns deslizes se repetem tanto entre iniciantes que merecem atenção redobrada: usar `=` no lugar de `==` dentro de uma condição — problema agravado pelo fato de o IntelliJ não avisar quando isso envolve um boolean; omitir as chaves `{}` no `if`, o que gera bugs silenciosos assim que uma nova linha é adicionada depois; e confundir `&&` com `||` — lembrando que AND exige que ambas as condições sejam verdadeiras, enquanto OR se contenta com apenas uma.

Todos os operadores deste capítulo, em um só lugar

```
= // Atribuição: int vagasDisponiveis = 10;
== // Igualdade: vagasDisponiveis == 10 → true
!= // Diferença
! // NOT: !true → false
&& // AND lógico: true && false → false
|| // OR lógico: true || false → true
?: // Ternário: String s = (vagasDisponiveis > 5) ? "muitas" : "poucas";
```

Fim de apostila

Com o domínio da lógica condicional e dos operadores booleanos, seu programa já consegue tomar decisões com base em dados reais.

O próximo passo natural nessa jornada são os laços de repetição e as estruturas de controle de fluxo mais avançadas.

JavaStudiesHub